

Accessing ADLS Gen2 from Azure Databricks

Using Access Key and Azure Key Vault–Backed Secret Scope

Project Setup & Screenshot Documentation

Data Engineering

September 2026

1. Project Overview

This document walks through the end-to-end setup of connecting Azure Databricks to Azure Data Lake Storage (ADLS) Gen2 / Blob Storage, using two authentication approaches: a direct Storage Account Access Key, and a more secure setup using Azure Key Vault–backed Databricks secret scopes. It is compiled from the step-by-step screenshots captured during the build.

Objectives

- Upload and verify source data files in an Azure Storage container.
- Review and manage Storage Account access keys.
- Provision an Azure Key Vault and configure its network access.
- Store sensitive values (storage keys) as Key Vault secrets.
- Grant the Databricks application the correct RBAC roles on the Key Vault.
- Create a Databricks-backed secret scope linked to the Key Vault.
- Provision a Databricks workspace, folder, and single-node compute cluster.
- Read data from Blob Storage / ADLS Gen2 into a Databricks notebook using both a SAS URL and a Spark configuration-based access key, and validate the output.

Security Note

- Storage account access keys and Key Vault secret values are highly sensitive. They have been masked or redacted in this document where they appeared in plain text on screen.
- Prefer the Key Vault–backed secret scope approach over hardcoding keys directly in notebooks — it keeps credentials out of code and version control.

2. Azure Blob Storage — Source Data Setup

The first step is preparing the storage account that will act as the data source (analogous to an ADLS Gen2 container). Two source files were uploaded and verified via the Storage Browser in the Azure Portal.

2.1 Verifying the Uploaded Source File

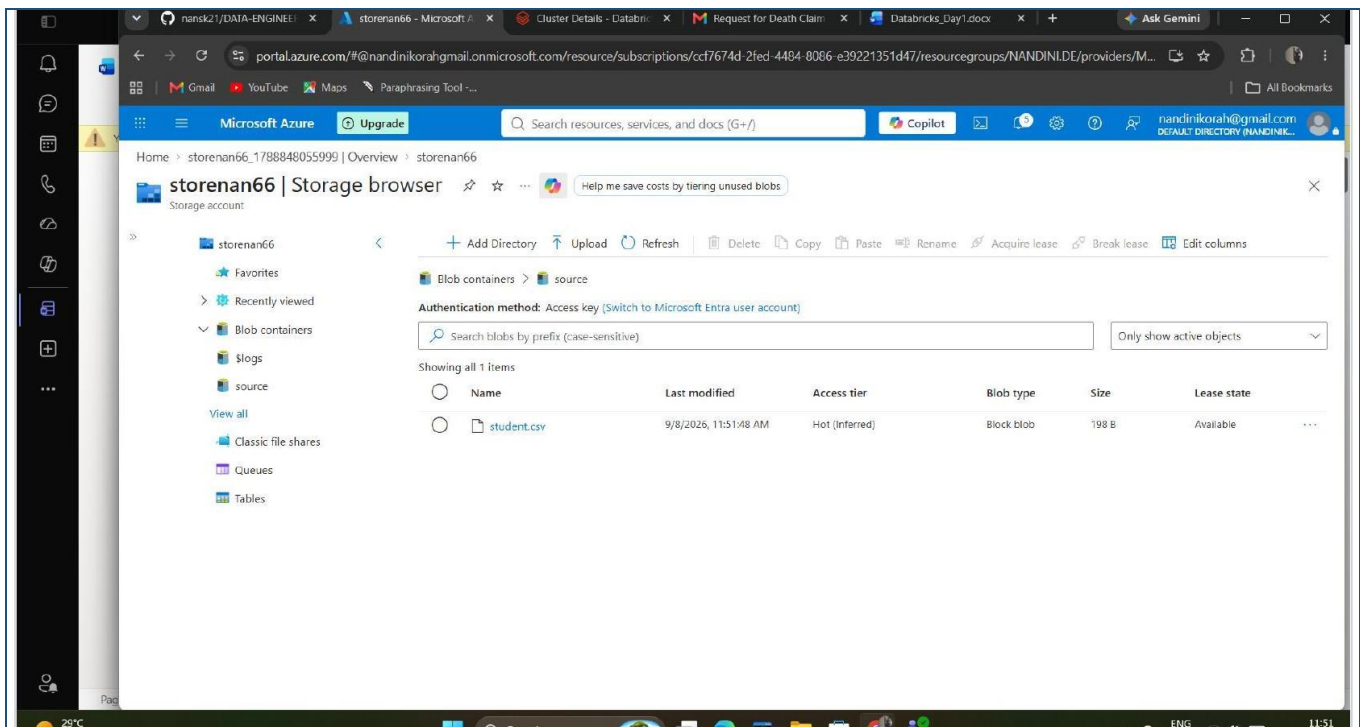


Fig. 1 — Azure Blob Storage container 'source' inside storage account storenan66, showing the uploaded student.csv file.

2.2 Verifying a Second Source File

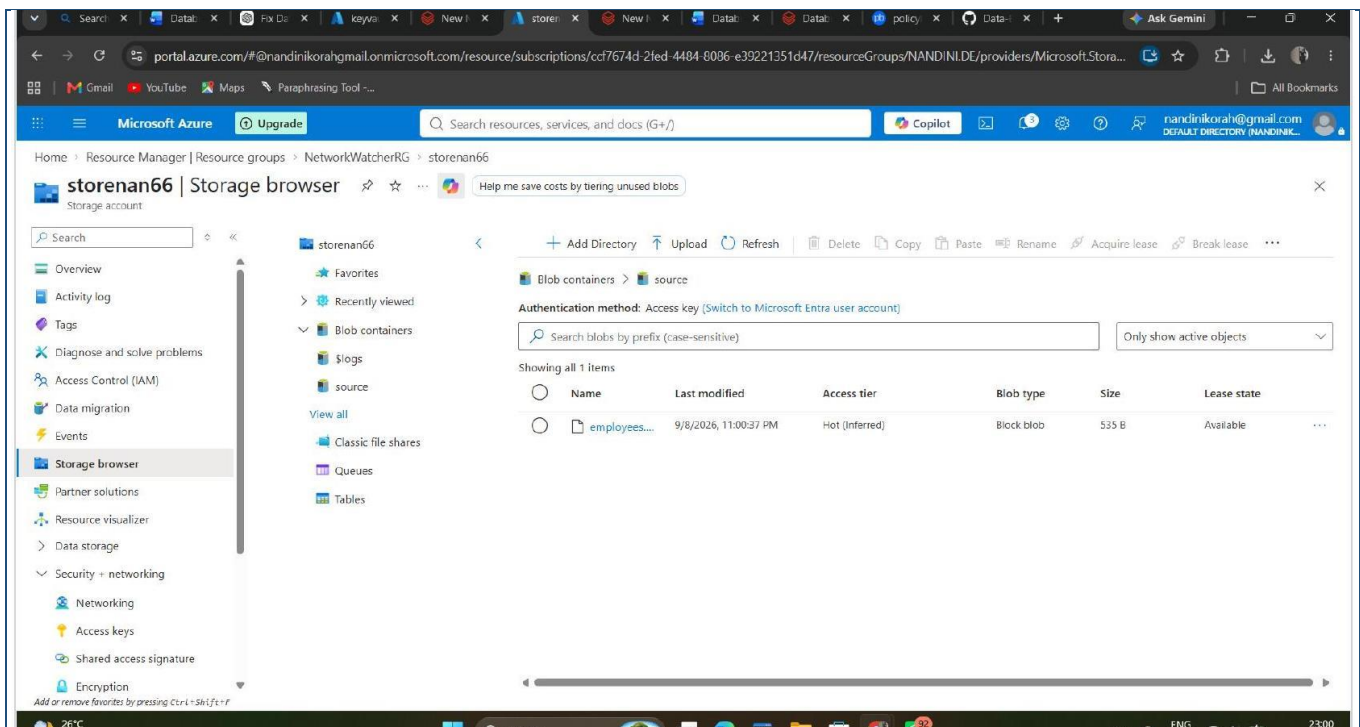


Fig. 2 — A second source file (employees...csv) confirmed inside the same 'source' container.

Key Point

- The Storage Browser confirms files exist in the 'source' blob container before any Databricks connection is attempted — this rules out data-availability issues later in the pipeline.

3. Authentication Method 1 — Storage Account Access Key

Azure Storage accounts expose two rotating access keys under Security + networking → Access keys. These keys grant full access to the storage account and can be used directly (e.g. via a SAS URL or a Spark configuration setting) or, preferably, stored securely in Key Vault.

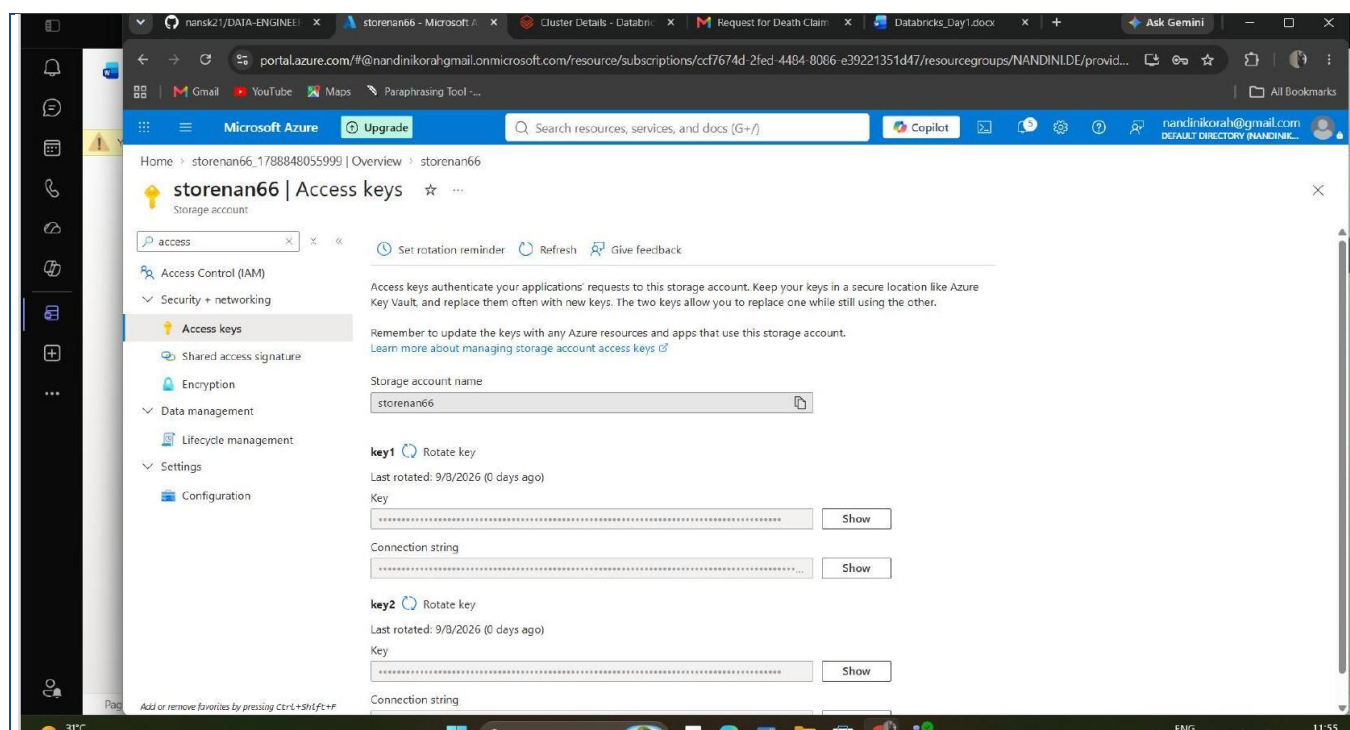


Fig. 3 — The Access keys blade for storage account storenan66, showing key1 and key2 (values hidden by default behind 'Show').

⚠ Security Warning

- Access keys grant full read/write control over the storage account. They should never be committed to notebooks, shared documents, or source control in plain text.
- Rotate keys regularly and prefer Key Vault-backed secrets or Microsoft Entra (Azure AD) authentication where possible.

4. Provisioning Azure Key Vault

To avoid hardcoding the storage access key, an Azure Key Vault (keyvaultnan66) was created to securely store secrets that Databricks can later retrieve through a secret scope.

4.1 Configuring Key Vault Network Access

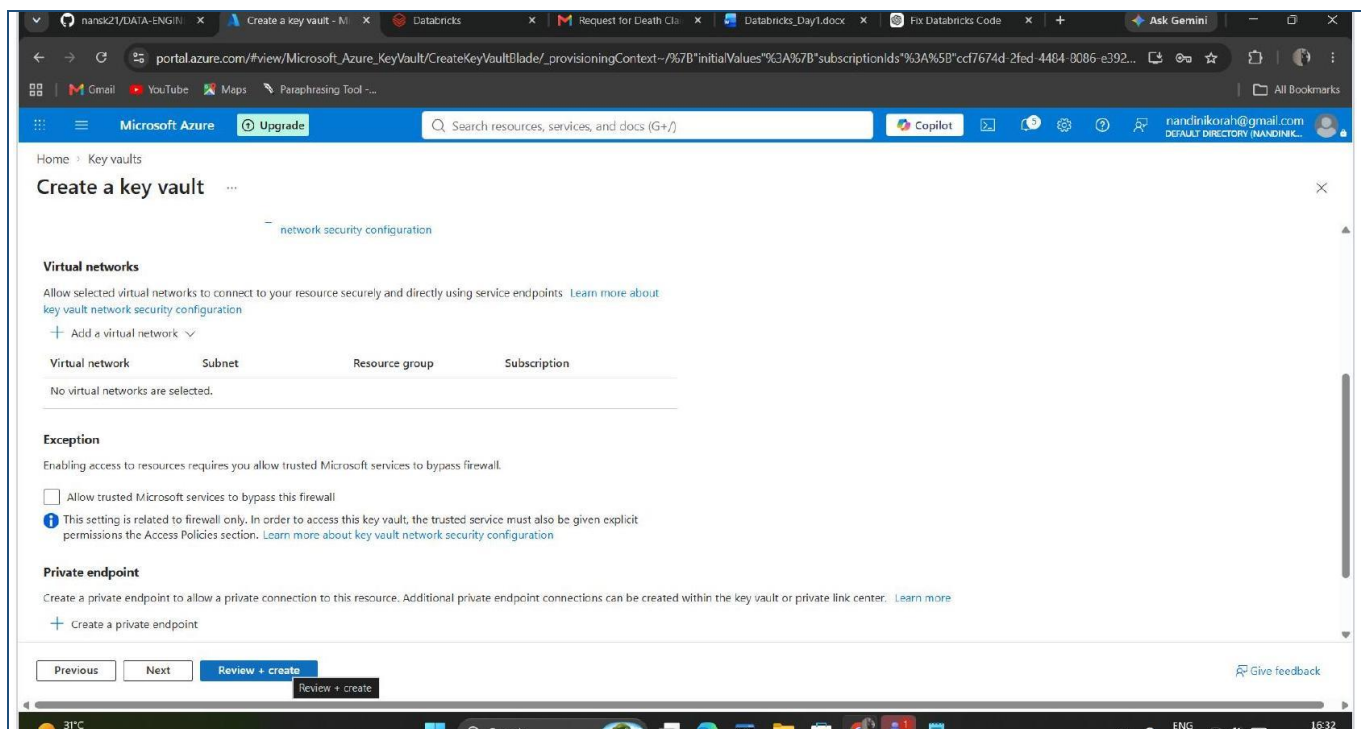


Fig. 4 — Reviewing virtual network and private endpoint options while creating the Key Vault.

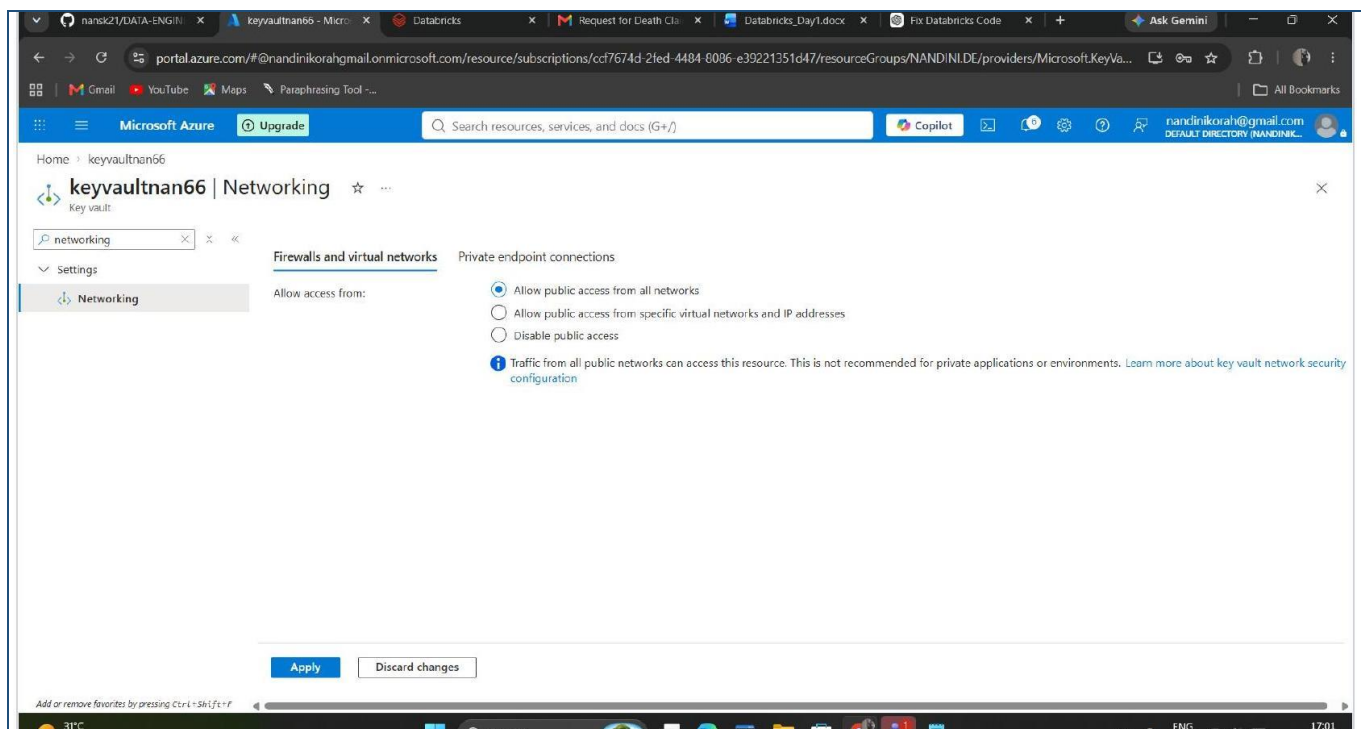


Fig. 5 — Key Vault networking configured to 'Allow public access from all networks' for this development setup.

⚠ Production Recommendation

- 'Allow public access from all networks' is convenient for a dev/lab environment but is not recommended for production.

- In production, restrict access using selected virtual networks/IP ranges, or disable public access entirely and use Private Endpoints.

4.2 Opening Key Vault Access Control (IAM)

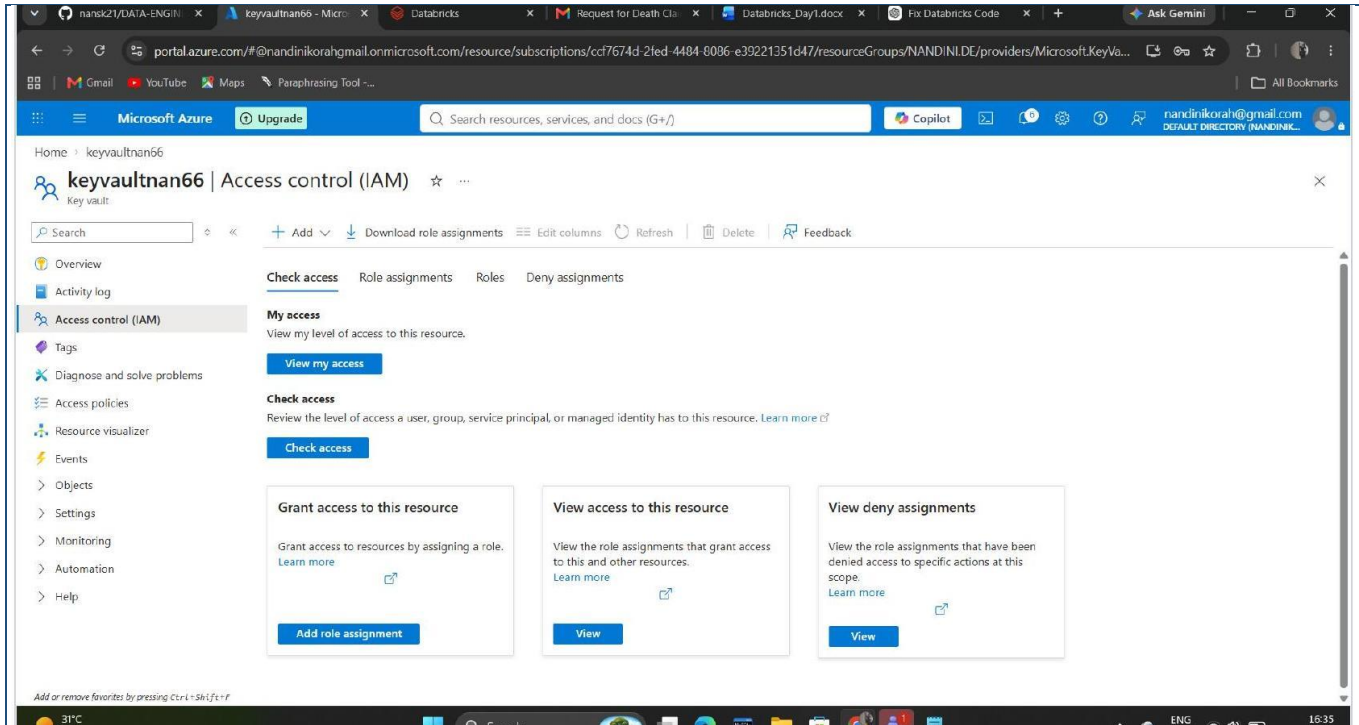


Fig. 6 — The Access control (IAM) blade for keyvaultnan66, used to manage role-based permissions on the vault.

5. Creating Key Vault Secrets

Multiple secrets were created inside the Key Vault to hold sensitive values (such as storage account keys) that Databricks notebooks will reference indirectly through a secret scope, instead of hardcoding them.

The screenshot shows the 'Create a secret' page in the Microsoft Azure portal. The form is filled out as follows:

- Upload options:** Manual
- Name:** secretdatabricksnan
- Secret value:** [Redacted]
- Content type (optional):** [Empty]
- Set activation date:** [Unchecked]
- Set expiration date:** [Checked]
- Expiration date:** 9/8/2028, 4:29:20 PM (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
- Enabled:** Yes
- Tags:** 0 tags

Buttons at the bottom: Create, Cancel, Search.

Fig. 7 — Creating the first secret ('secretdatabricksnan') with an expiration date set.

The screenshot shows the 'Create a secret' page in the Microsoft Azure portal. The form is filled out as follows:

- Upload options:** Manual
- Name:** employeesecret
- Secret value:** [Redacted]
- Content type (optional):** [Empty]
- Set activation date:** [Unchecked]
- Set expiration date:** [Unchecked]
- Enabled:** Yes
- Tags:** 0 tags

Buttons at the bottom: Create, Cancel.

Fig. 8 — Creating a second secret ('employeesecret').

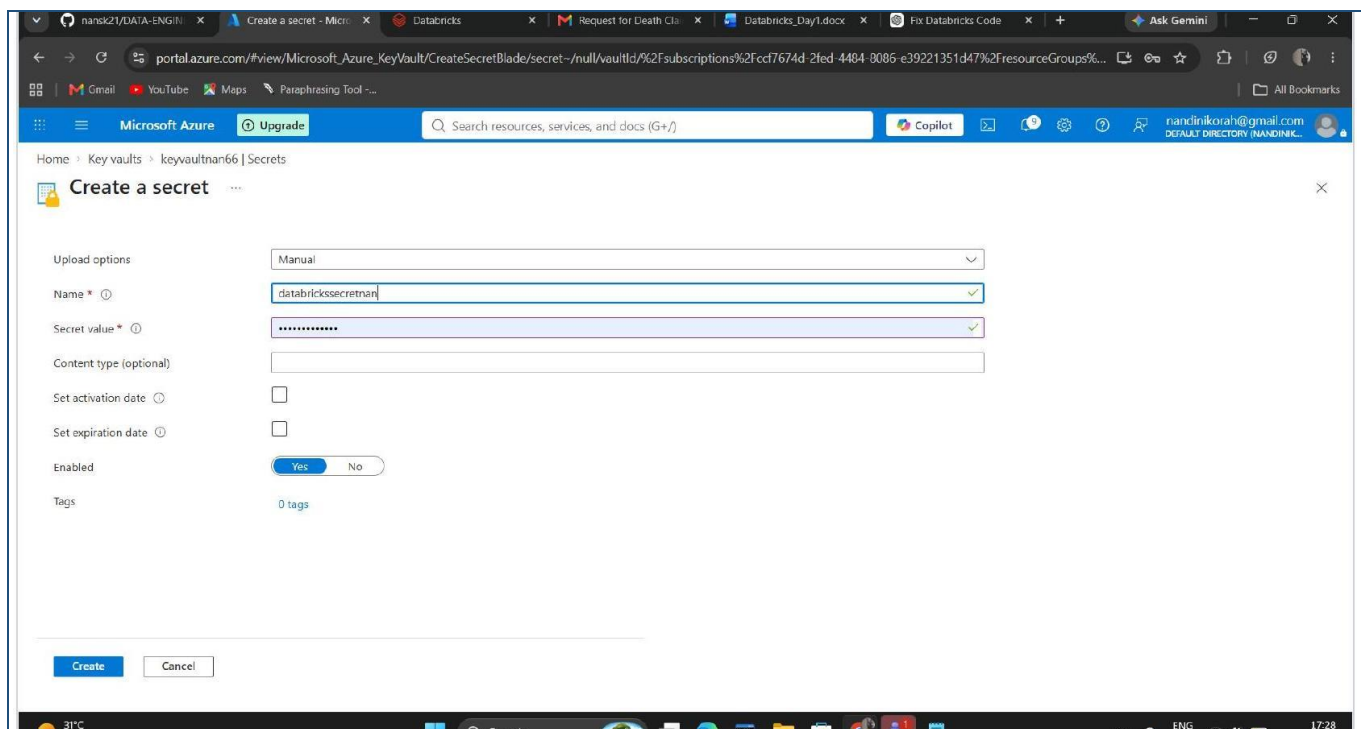


Fig. 9 — Creating a third secret ('databrickssecretnan') to hold the storage access key value.

Key Point

- Secret values are entered as masked input and are never displayed in plain text in the portal after creation — they can only be revealed via 'Show' by an authorized principal.

6. Granting Key Vault Access via RBAC Role Assignments

For Databricks to read secrets from the Key Vault when a secret scope is created, both the user and the Databricks Resource Provider application need appropriate RBAC roles on the vault.

6.1 Selecting the Role

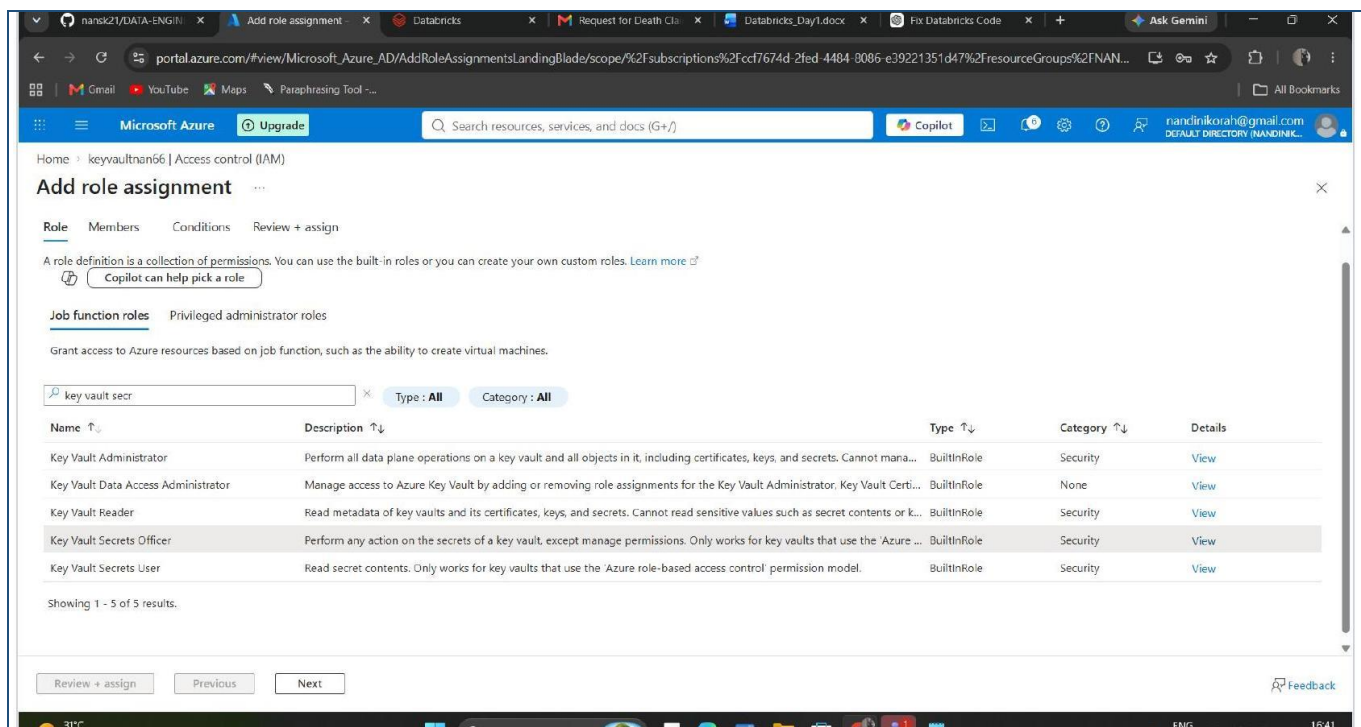


Fig. 10 — Reviewing built-in Key Vault roles (Administrator, Data Access Administrator, Reader, Secrets Officer, Secrets User).

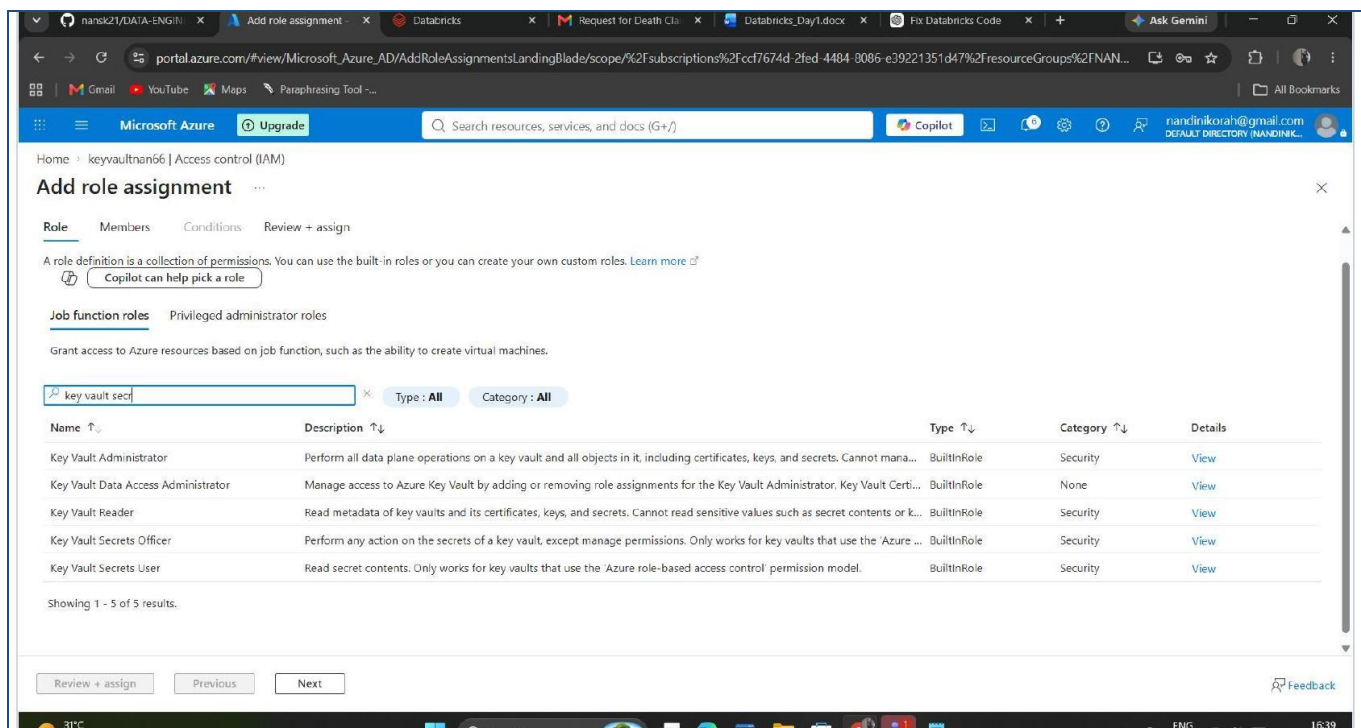


Fig. 11 — Selecting the 'Key Vault Secrets Officer' role, which allows managing (but not granting access to) secrets.

⚠ Key Point — Role Choice

- Key Vault Secrets Officer: can perform any action on secrets, except manage permissions — assigned to the user for administration.

- Key Vault Secrets User: read-only access to secret contents — assigned to the Databricks Resource Provider so the platform can retrieve values for a secret scope.

6.2 Assigning Members to the Role

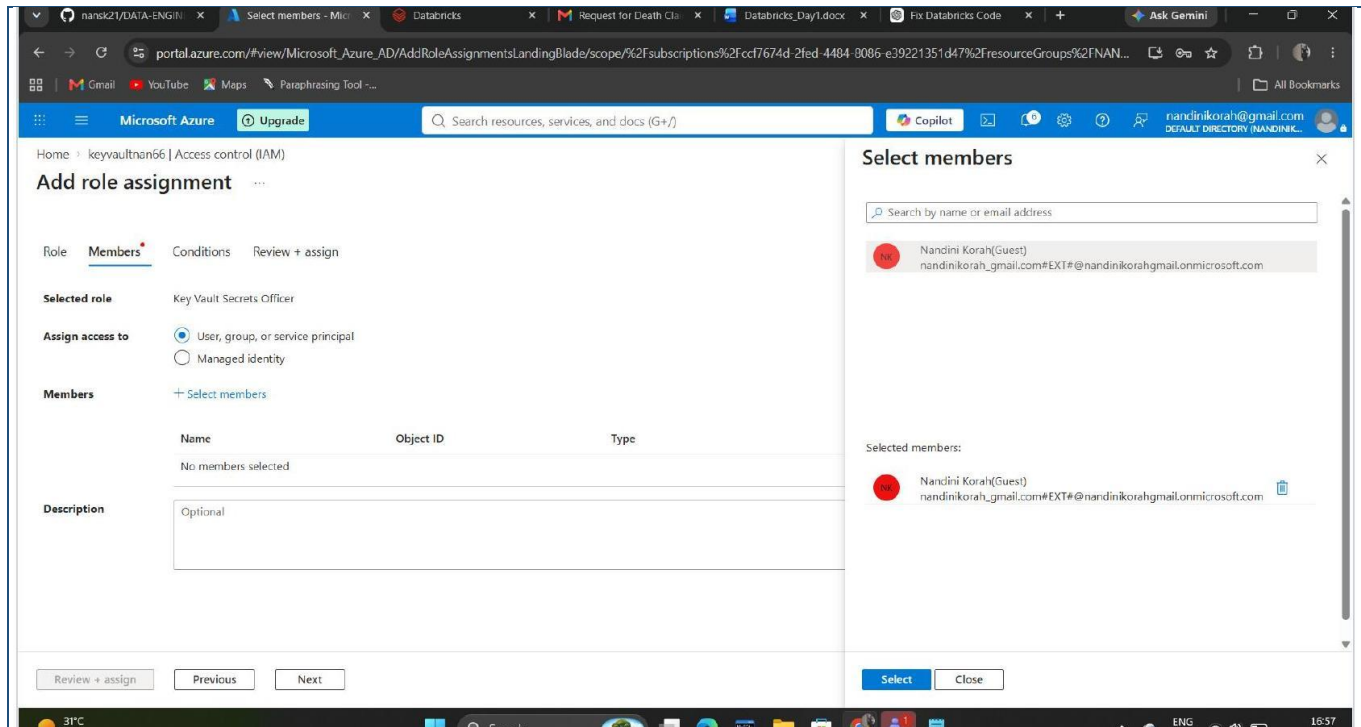


Fig. 12 — Adding the user (Nandini Korah) as a member for the 'Key Vault Secrets Officer' role.

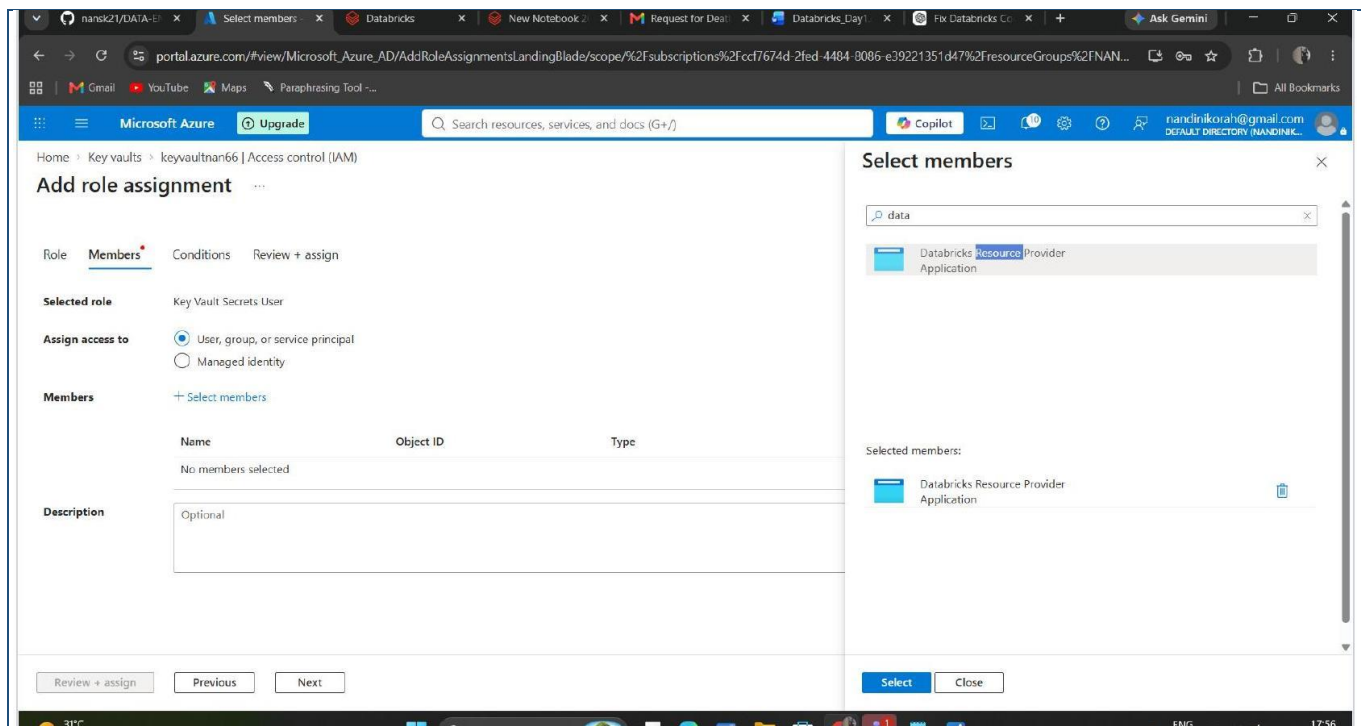


Fig. 13 — Adding the 'Databricks Resource Provider' application as a member for the 'Key Vault Secrets User' role, enabling Databricks to read secrets.

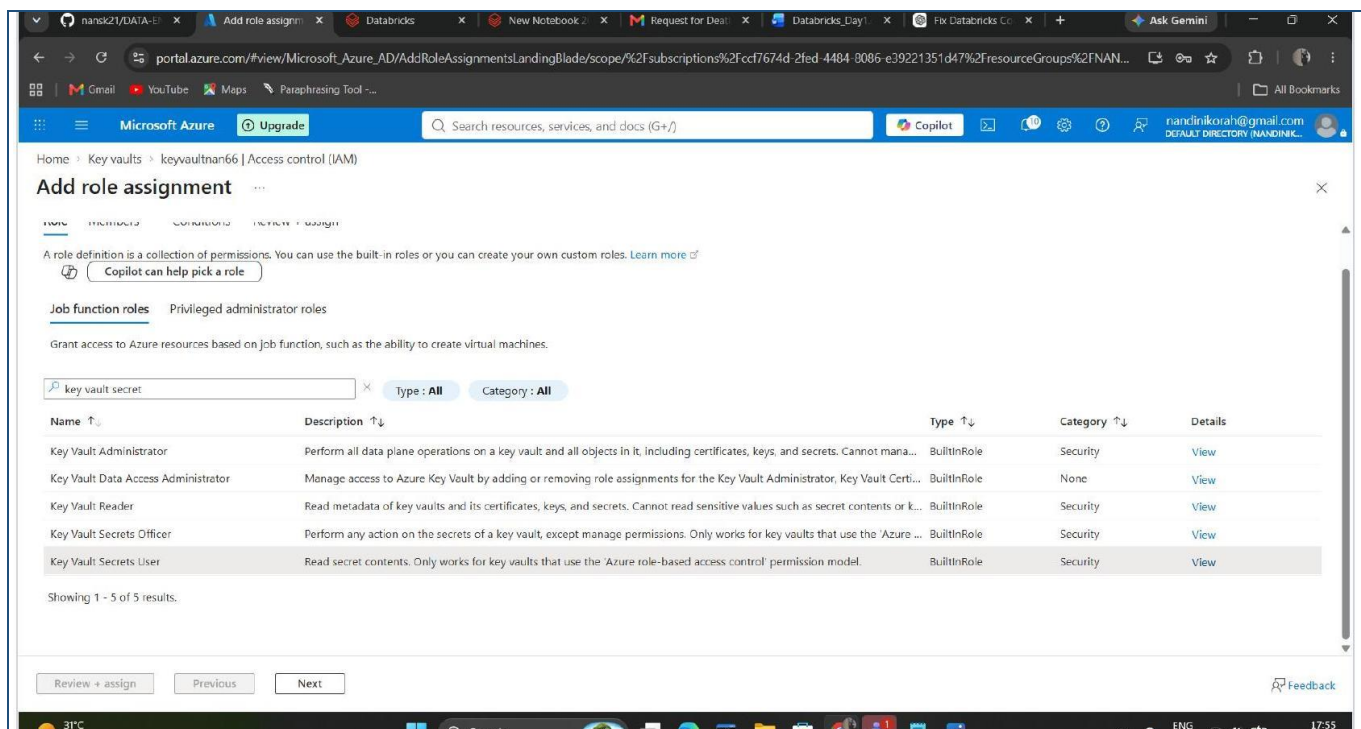


Fig. 14 — Final review of the available Key Vault role assignment options before confirming.

7. Databricks Workspace Setup

7.1 Creating the Workspace Folder

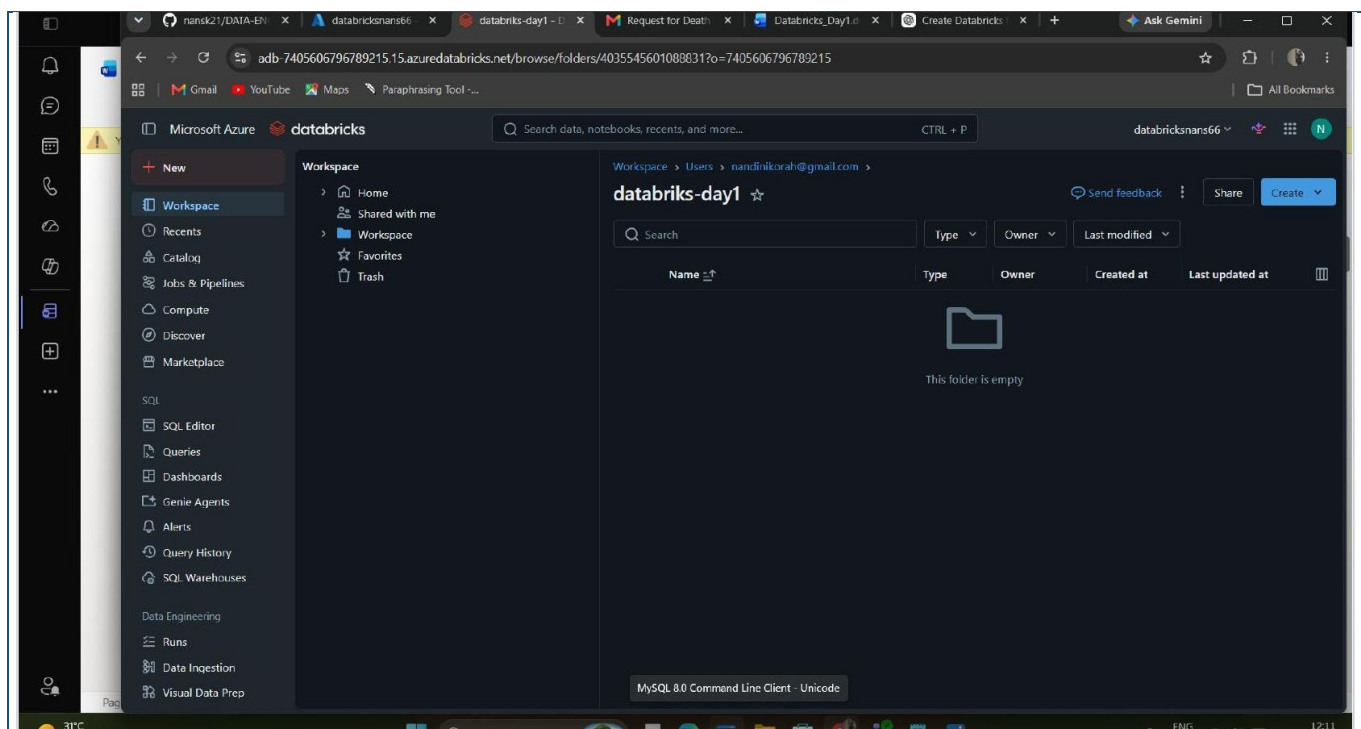


Fig. 15 — A new empty workspace folder 'databricks-day1' created under the user's home directory in Databricks.

7.2 Creating a Databricks-Backed Secret Scope

A secret scope links Databricks to the Azure Key Vault, so notebooks can retrieve secrets using `dbutils.secrets.get()` instead of hardcoded values.

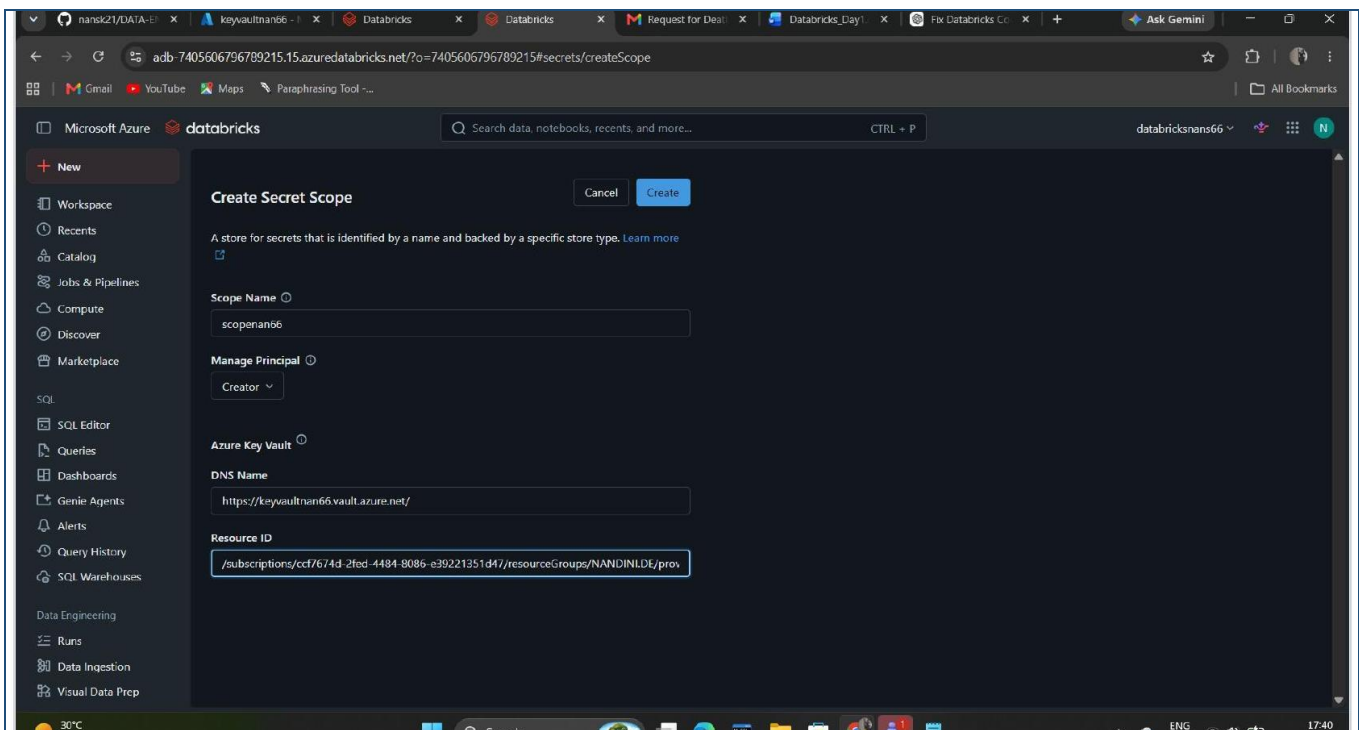


Fig. 16 — Creating the secret scope 'scopenan66', pointing to the Key Vault DNS name and Resource ID of keyvaultnan66.

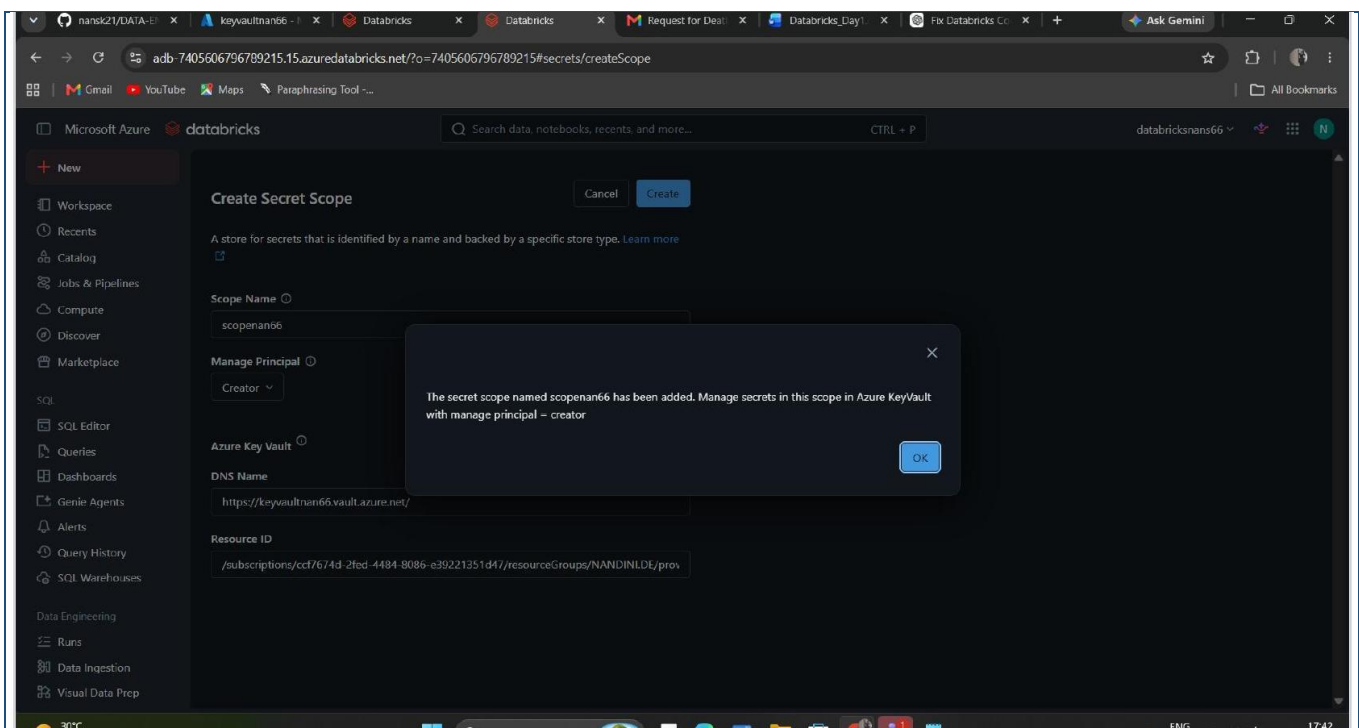


Fig. 17 — Confirmation that the secret scope 'scopenan66' was successfully created, with manage principal = creator.

Key Point

- Once the secret scope is created, any secret stored in the linked Key Vault (e.g. databrickssecretname) becomes retrievable inside notebooks via `dbutils.secrets.get(scope='scopenan66', key='...')` — without ever exposing the raw value in code.

7.3 Provisioning Compute

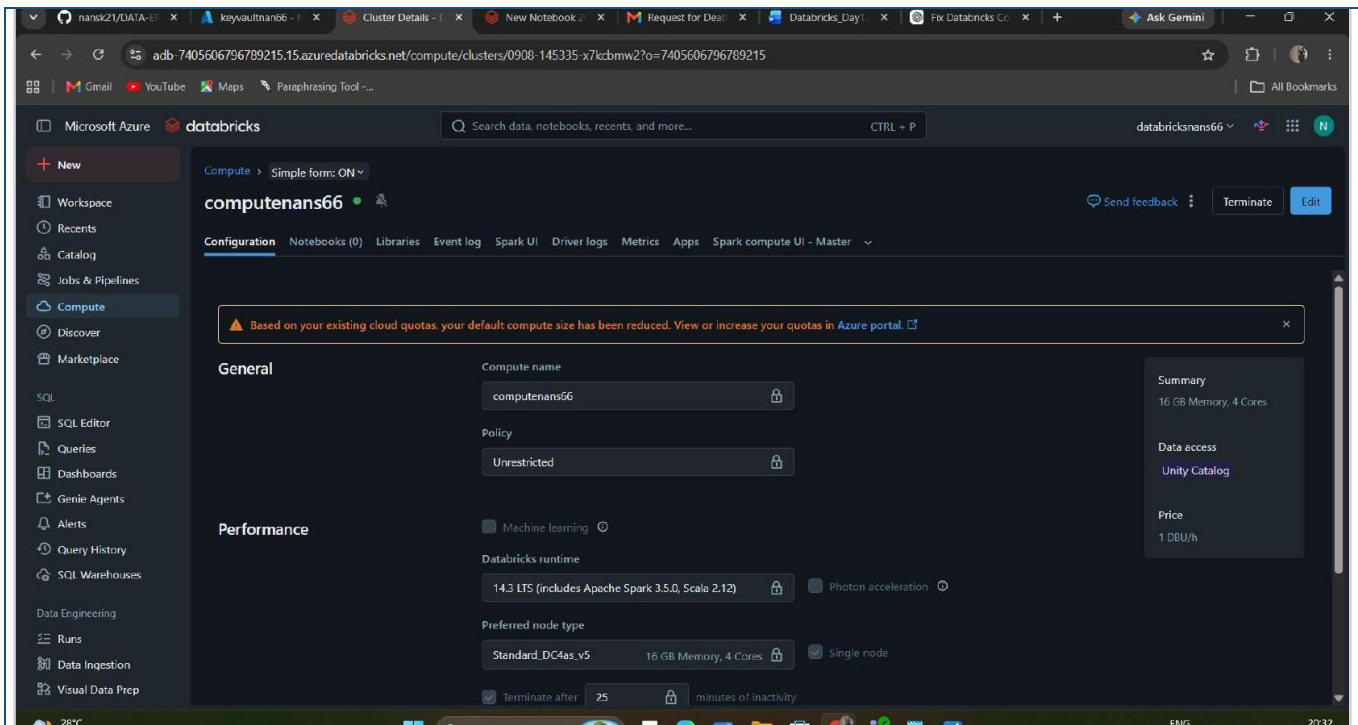


Fig. 18 — Configuring a new single-node compute resource (computenans66): Databricks runtime 14.3 LTS, Standard_DC4as_v5 node, Unity Catalog data access, auto-terminate after 25 minutes.

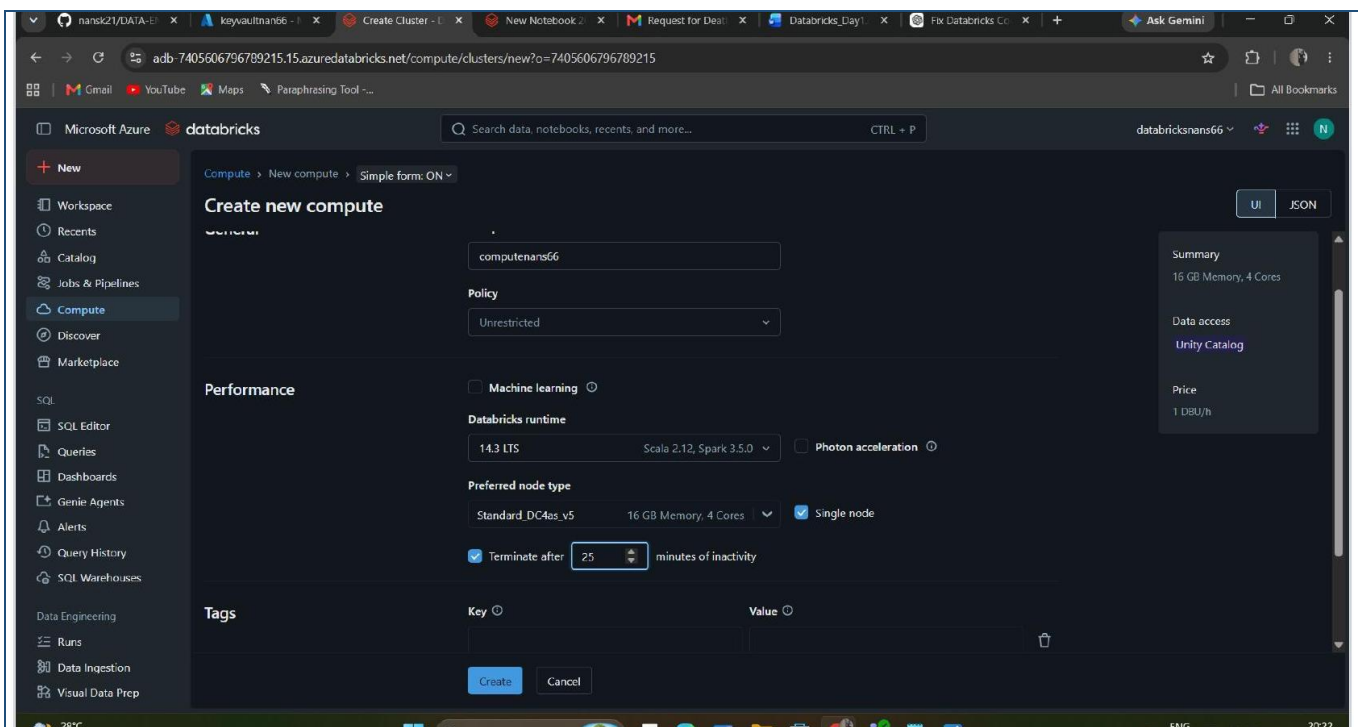


Fig. 19 — The 'Create new compute' JSON/UI form showing the finalized cluster configuration before creation.

8. Reading Data in a Databricks Notebook

8.1 Creating the Notebook

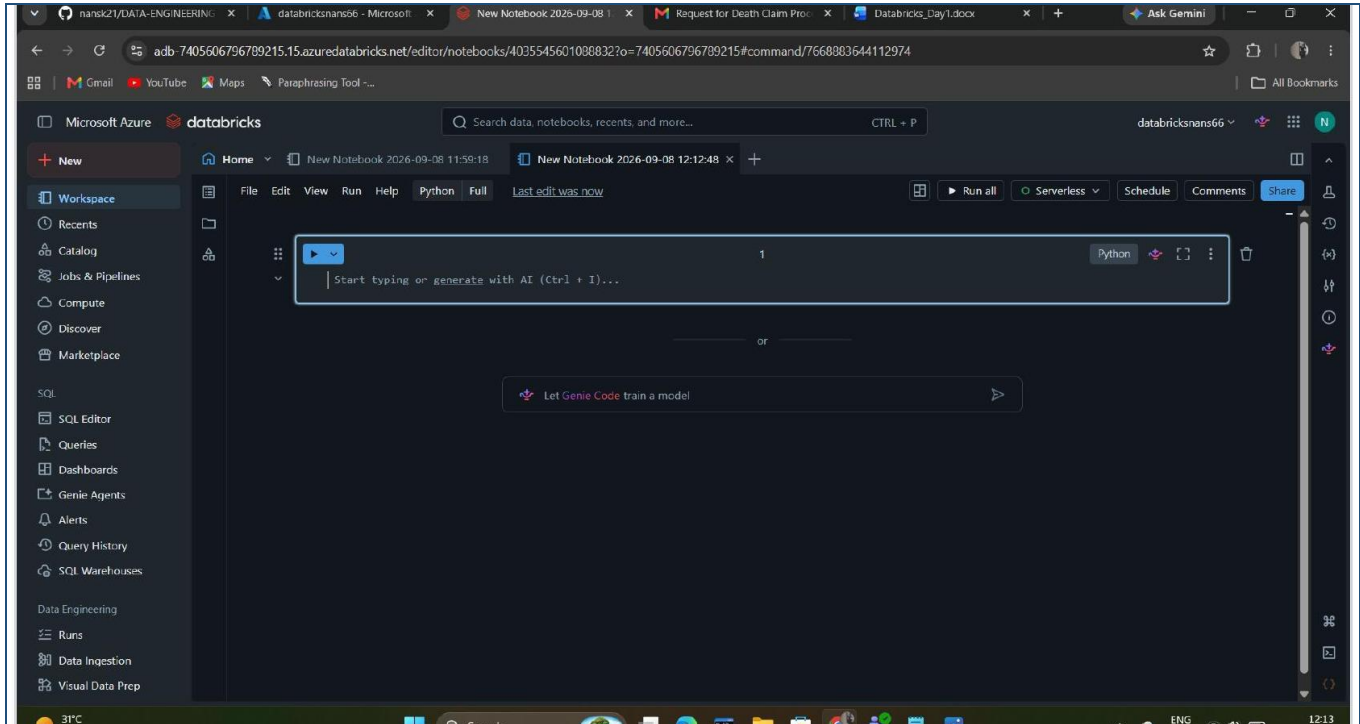


Fig. 20 — A new Python notebook created inside the Databricks workspace, ready for the data-access code.

8.2 Access Method A — SAS URL

The notebook first reads student.csv using a time-limited Shared Access Signature (SAS) URL generated for the blob, loading it into a pandas DataFrame and then converting it to a Spark DataFrame.

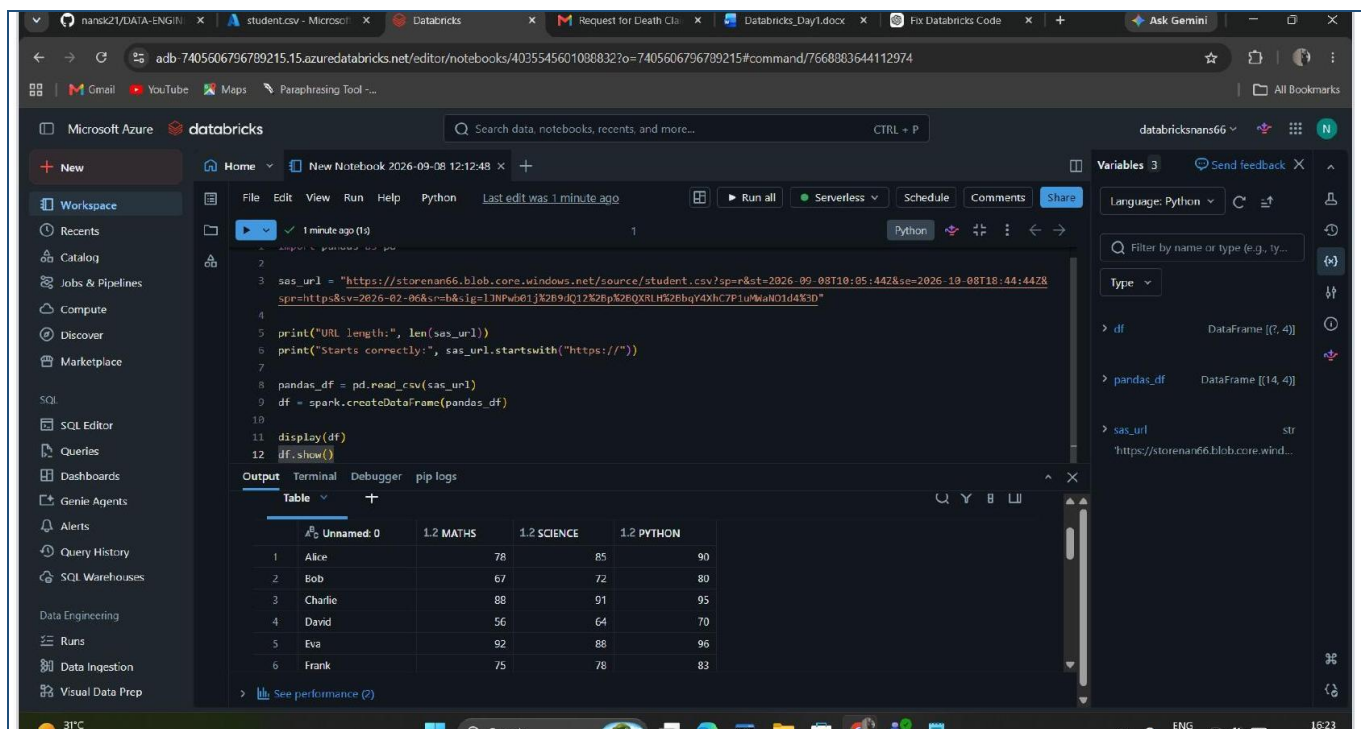


Fig. 21 — Notebook code using a SAS URL with `pd.read_csv()` and `spark.createDataFrame()`, with `df.show()` output displaying 14 rows.

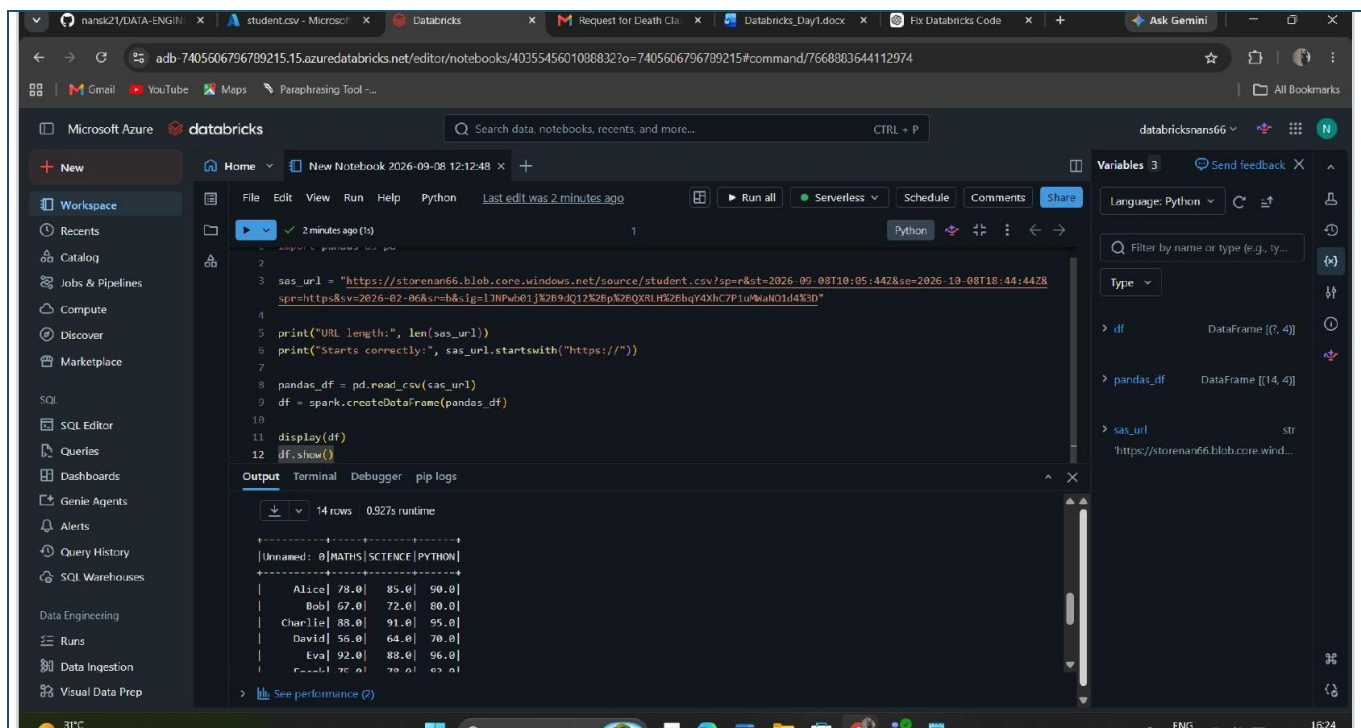


Fig. 22 — Re-running the notebook to confirm consistent output: Alice, Bob, Charlie, David, Eva and other rows with MATHS, SCIENCE and PYTHON scores.

⚠ Security Note — SAS URL

- The SAS URL embeds a signed access token with a defined start/expiry window (visible in the code as 'st=' and 'se=' parameters). It should be treated as a secret and regenerated / rotated periodically.

8.3 Access Method B — Spark Configuration with Storage Access Key

As an alternative (and closer to a typical ADLS Gen2 access-key pattern), the storage account key is set directly via `spark.conf.set()`, and the file is then read from the `abfss://` path.

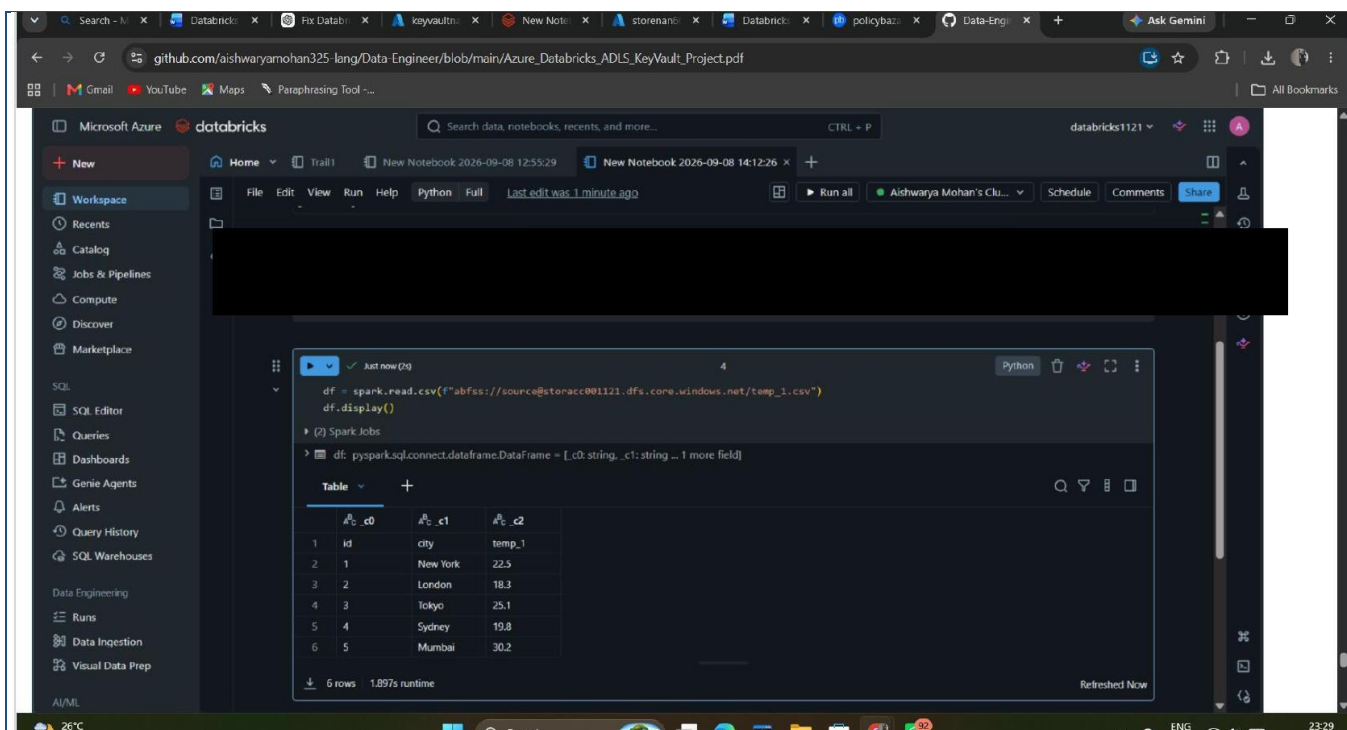


Fig. 23 — Notebook using `spark.conf.set('fs.azure.account.key...')` followed by `spark.read.csv()` on an `abfss://` path, showing 6 rows of output (`id`, `city`, `temp_1`). The access key value has been redacted for security.

⚠ Best Practice — Avoid Hardcoded Keys

- Hardcoding the storage key directly in `spark.conf.set()` (as shown above) works but exposes the credential in the notebook and its revision history.
- The preferred approach is to retrieve the key at runtime from the Key Vault-backed secret scope, e.g.: `spark.conf.set("fs.azure.account.key.<account>.dfs.core.windows.net", dbutils.secrets.get(scope="scopenan66", key="databrickssecretnan"))`.

9. Summary

This project demonstrated two ways to connect Azure Databricks to Azure Storage (ADLS Gen2 / Blob):

1. Direct Access Key — quick to set up, but the key must be protected carefully (e.g. via SAS URLs or Spark config) and rotated regularly.
2. Key Vault-Backed Secret Scope — the recommended pattern: the storage key is stored once as a Key Vault secret, appropriate RBAC roles (Key Vault Secrets Officer / Key Vault Secrets User) are granted, a Databricks secret scope is linked to the vault, and notebooks retrieve the key at runtime via `dbutils.secrets.get()` rather than embedding it in code.

The workspace, compute cluster, and notebooks were verified end-to-end by successfully reading and displaying source CSV data from Blob Storage in Databricks using both methods.

Overall Recommendation

- For any environment beyond a personal sandbox, standardize on the Key Vault-backed secret scope approach and avoid embedding access keys in notebook code.
- Consider Microsoft Entra ID (Azure AD) passthrough or a Service Principal with Unity Catalog external locations as an even more modern, keyless alternative to storage account keys.